> Neka je dat pozitivan realan broj x sa konačnim decimalskim zapisom i neka su dati prirodni brojevi n i m, tako da $n<m$. Formirati niz razlomaka p/q takvih da za imenilac q važi $n<=q<=m$ (tj. $q=n,n+1,...,m$) i pri tom imeniocu q pridružujemo brojilac p koji određujemo zaokruživanjem na najbliži priodan broj proizvoda $x \cdot q$. Predstaviti svaki razlomak p/q u obliku verižnog razlomaka. U nizu razlomaka p/q izdvojiti:
>
> - najbolje racionalne aproksimacije I vrste,
> - najbolje racionalne aproksimacije II vrste,
> - sortirati sve razlomke p/q po uslovu minimalnosti apsolutne greske $|x-p/q|$.

Napisani kod ćemo testirati na primjeru konstante $v=\pi-3=0.14159265359...$, a za parametre n i m uzeti brojeve 5 i 30 respektivno.

In [11]:
```
from math import pi

v = pi - 3
n, m = 5, 30
```

Prvi korak je pisanje pomoćnih funkcija, koje će olakšati generisanje verižnog oblika razlomaka i utvrđivanje "kvaliteta" razlomaka.

Prva funkcija (*generate_continued_fraction*) će generisati verižni oblik razlomka za dato p i q. Naredne dvije funkcije (*is_fst_deg_best* i *is_snd_deg_best*) utvrđuju da li se radi o najboljoj racionalnoj aproksimaciji I i II vrste.

In [12]:
```
def generate_simple_continued_fraction(p: int, q: int) -> list[int]:
        a = [0]
        a[0], p = divmod(p, q)

        while p != 0:
                quotient, remainder = divmod(q, p)
                a.append(quotient)
                p, q = remainder, p

        return a


def is_fst_deg_best(x: float, p: int, q: int) -> bool:
        for s in range(1, q):
                r = round(s * x)
                if r / s == p / q:
                        continue
```

```
                if abs(x - r / s) < abs(x - p / q):
                        return False

        return True

def is_snd_deg_best(x: float, p: int, q: int) -> bool:
        for s in range(1, q):
                r = round(s * x)
                if r / s == p / q:
                        continue

                if abs(s * x - r) < abs(q * x - p):
                        return False

        return True
```

Nakon pisanja pomoćnih funkcija, možemo preći na dio koda koji prolazi kroz svako moguće q i generiše razlomke. A usput, za svaki razlomak, utvrđujemo da li se radi o najboljoj racionalnoj aproksimaciji.

Sve razlomke ćemo čuvati u listi, kako bismo ih mogli kasnije i sortirati i ispisati.

In [13]:
```
from dataclasses import dataclass

@dataclass(order=True)
class RationalApproximation:
        p: int
        q: int
        a: list[int]

        first_degree: bool
        second_degree: bool

        def __float__(self) -> float:
                return self.p / self.q

fractions: list[RationalApproximation] = []
for q in range(n, m + 1):
        p = round(q * v)

        a = generate_simple_continued_fraction(p, q)
        first_degree = is_fst_deg_best(v, p, q)
        second_degree = is_snd_deg_best(v, p, q) if first_degree else False

        fractions.append(RationalApproximation(
```

```
                p, q,
                a, first_degree, second_degree
        ))
```

Potom je potrebno isfiltrirati i sortirati razlomke. Prema tekstu zadatka, sortiranje se vrši po rastućoj vrijednosti apsolutne greške. Dok, kada je riječ o filtriranju, potrebno je ukloniti razlomke koji su u suštini isti, tj. one koji se pojavljuju i u nekom skraćenom obliku (npr. *7/12* i *14/24*).

In [14]:
```
def simplify_fraction(p: int, q: int) -> tuple[int, int]:
    i = 2
    while i < min(p, q) + 1:
        if p % i == 0 and q % i == 0:
            p = p // i
            q = q // i
        else:
            i += 1
    return p, q

unique_fractions: dict[tuple[int, int], list[RationalApproximation]] = {}
for frac in fractions:
    simplified = simplify_fraction(frac.p, frac.q)
    if simplified not in unique_fractions:
        unique_fractions[simplified] = []
    unique_fractions[simplified].append(frac)

fractions = [min(group) for group in unique_fractions.values()]
fractions.sort(key=lambda r: abs(r.p / r.q - v))
```

Ssada imamo sve informacije da ispišemo traženu tabelu. Svaki razlomak je napisan u 3 oblika:

1. kao razlomak
2. kao realan broj
3. kroz verižni zapis

Različite boje određuju "kvalitet" aproksimacije:

- **Crvena** boja - Najbolja racionalna aproksimacija I vrste
- **Plava** boja - Najbolja racionalna aproksimacija II vrste
- **Bijela** boja - Ostali razlomci

In [15]:
```
from termcolor import colored

def format_simple_continued_fraction(a: list[int]) -> str:
```

```
        if len(a) == 1:
                return f"[{a[0]}]"

        return f"[{a[0]}; {', '.join(map(str, a[1:]))}]"

for fraction in fractions:
        if fraction.second_degree:
                color = "red"
        elif fraction.first_degree:
                color = "blue"
        else:
                color = "light_grey"

        ratio_format = f"{fraction.p}/{fraction.q}"
        real_format = f"{float(fraction):.9f}"
        difference = f"{v - float(fraction):+.9f}"
        array_format = format_simple_continued_fraction(fraction.a)

        text = f"{ratio_format} = {real_format} ({difference}), {array_format}"

        print(colored(text, color))
```

```
1/7 = 0.142857143 (-0.001264489), [0; 7]
4/29 = 0.137931034 (+0.003661619), [0; 7, 4]
3/22 = 0.136363636 (+0.005229017), [0; 7, 3]
4/27 = 0.148148148 (-0.006555495), [0; 6, 1, 3]
2/15 = 0.133333333 (+0.008259320), [0; 7, 2]
3/20 = 0.150000000 (-0.008407346), [0; 6, 1, 2]
3/23 = 0.130434783 (+0.011157871), [0; 7, 1, 2]
2/13 = 0.153846154 (-0.012253500), [0; 6, 2]
3/19 = 0.157894737 (-0.016302083), [0; 6, 3]
1/8 = 0.125000000 (+0.016592654), [0; 8]
4/25 = 0.160000000 (-0.018407346), [0; 6, 4]
2/17 = 0.117647059 (+0.023945595), [0; 8, 2]
1/6 = 0.166666667 (-0.025074013), [0; 6]
1/9 = 0.111111111 (+0.030481542), [0; 9]
2/11 = 0.181818182 (-0.040225528), [0; 5, 2]
1/10 = 0.100000000 (+0.041592654), [0; 10]
1/5 = 0.200000000 (-0.058407346), [0; 5]
```